Московский государственный университет
имени М. В. Ломоносова
Механико-математический факультет

Отчет по заданию № 2 о решении задачи оптимального управления

Содержание

1. Постановка задачи . . . 2
2. Формализация задачи . . . 2
3. Система необходимых условий оптимальности . . . 2
4. Краевая задача . . . 3
5. Метод стрельбы . . . 3
6. Метод Рунге-Кутты четвертого порядка . . . 4
7. Метод Ньютона для решения нелинейной системы . . . 4
8. Аналитическое решение для $\alpha = 0$ . . . 5
9. Результаты программы . . . 6
10. Продолжение по параметру . . . 8
11. Вывод . . . 8
12. Литература . . . 8
13. Листинг программы . . . 9

1 Постановка задачи

Рассматривается задача оптимального управлением:

$$J[x] = \int_0^1 \left(\ddot{x}^2 - \frac{48x}{1+e^{-\alpha t}} \right) dt \to \text{extr} \tag{1}$$

с граничными условиями:

$$x(1) = 0, \quad \dot{x}(0) = 0 \tag{2}$$

и параметрическим исследованием при:

$$\alpha \in \{0.0, 0.1, 1.0, 5.1\}$$

2 Формализация задачи

Приведем исходную задачу к стандартной задаче оптимального управления. Для этого положим:

$$\begin{cases} x_1 = x; \\ x_2 = \dot{x} \\ u = \ddot{x}. \end{cases} \tag{3}$$

Тогда исследуемая задача в исследуемых обозначениях перепишется следующим образом:

$$J[x] = \int_0^1 \left(u^2 - \frac{48x_1}{1+e^{-\alpha t}} \right) dt \to \text{extr} \tag{4}$$

$$x_1(1) = 0, \quad x_2(0) = 0 \tag{5}$$

3 Система необходимых условий оптимальности

Составим функцию Лагранжа

$$J = \int_0^1 L \; dt + l, \tag{6}$$

где

$$L = \lambda_0 (u^2 - \frac{48x_1}{1+e^{-\alpha t}}) + p_1(\dot{x}_1 - x_2) + p_2(\dot{x}_2 - u) \tag{7}$$

— лагранжиан, а

$$l = \lambda_1 x_1(1) + \lambda_2 x_2(0) \tag{8}$$

— терминант. Здесь λ_0, λ_1, λ_2, $p_1 = p_1(t)$, $p_2 = p_2(t)$ — числовые и функциональные множители Лагранжа. Также нам понадобится функция Понтрягина

$$H = p_1 x_2 + p_2 u - \lambda_0 (u^2 - \frac{48x_1}{1+e^{-\alpha t}}) \tag{9}$$

Теперь запишем систему необходимых условий оптимальности в задаче (3)–(6).

1. Уравнения Эйлера — Лагранжа:

$$-\frac{d}{d\,t} L_{\dot{x}} + L_x = 0 \Leftrightarrow \begin{cases} -\dot{p}_1 - \lambda_0 \frac{48}{1+e^{-\alpha t}} = 0; \\ -\dot{p}_2 - p_1 = 0. \end{cases}$$

2. Условия трансверсальности:

$$L_{\dot{x}}(0) = l_{x(0)} \Leftrightarrow \begin{cases} p_1(0) = 0; \\ p_2(0) = \lambda_2, \end{cases} \qquad L_{\dot{x}}(1) = -l_{x(1)} \Leftrightarrow \begin{cases} p_1(1) = -\lambda_1; \\ p_2(1) = 0. \end{cases}$$

3. Условие оптимальности по управлению:

$$\frac{\partial H}{\partial u} = 0 \quad \Rightarrow \quad p_2 - 2\lambda_0 u = 0 \quad \Rightarrow \quad \hat{u} = \frac{p_2}{2\lambda_0}$$

4. Множители Лагранжа λ_0, λ_1, λ_2, λ_3 не равны одновременно нулю.

5. Условие неотрицательности: $\lambda_0 \geq 0$.

6. Условие нормировки: множители Лагранжа λ_0, λ_1, λ_2, λ_3 могут быть выбраны с точностью до положительного сомножителя.

4 Краевая задача

Для формирования краевой задачи принципа максимума необходимо выбрать условие нормировки. Проведем анализ возможности аномального случая $\lambda_0 = 0$. В этой ситуации $\dot{p}_1 = 0$ и $\dot{p}_2 = -p_1$, откуда получаем $p_1 = C_1$ и $p_2 = C_1 t + C_2$, где $C_1 C_2$ - некоторые константы. Поскольку $p_1(0) = 0$ и $p_2(1) = 0$, то $p_1 = p_2 = 0$, что влечёт $\lambda_1 = \lambda_2 = \lambda_3 = 0$, что противоречит нашим условиям, отсюда $\lambda_0 > 0$.

В качестве условия нормировки возьмем $\lambda_0 = \frac{1}{2}$. Тогда рассматриваемая задача оптимального управления сводится к решению следующей краевой задачи:

$$\begin{cases} \dot{x}_1 = x_2; \\ \dot{x}_2 = p_2; \\ \dot{p}_1 = -\frac{24}{1+e^{-\alpha t}}; \\ \dot{p}_2 = -p_1 \\ x_1(1) = 0 \\ x_2(0) = 0 \\ p_1(0) = 0 \\ p_2(1) = 0 \end{cases} \tag{10}$$

5 Метод стрельбы

Краевая задача (10) решается методом стрельбы. В качестве параметров пристрелки выбираются недостающие для решения задачи Коши значения при $t = 0$, а именно $x_1(0) = \beta_1$ и $p_2(0) = \beta_2$. Задав эти значения, получаем задачу Коши:

$$\begin{cases} \dot{x}_1 = x_2; \\ \dot{x}_2 = p_2; \\ \dot{p}_1 = -\dfrac{24}{1+e^{-\alpha t}}; \\ \dot{p}_2 = -p_1; \\ x_1(0) = \beta_1; \\ x_2(0) = 0; \\ p_1(0) = 0; \\ p_2(0) = \beta_2. \end{cases} \tag{11}$$

Решая задачу (11) методом Рунге-Кутты четвертого порядка, получаем:

$$\begin{cases} x_1[\beta_1, \beta_2]; \\ x_2[\beta_1, \beta_2]; \\ p_1[\beta_1, \beta_2]; \\ p_2[\beta_1, \beta_2]. \end{cases}$$

Для решения краевой задачи необходимо подобрать такие значения параметров β_1 и β_2, что

$$\begin{cases} x_1(1)[\beta_1, \beta_2] = 0; \\ p_2(1)[\beta_1, \beta_2] = 0. \end{cases}$$

Таким образом, решение краевой задачи свелось к решению системы двух функциональных уравнений от двух неизвестных.

6 Метод Рунге-Кутты четвертого порядка

Для решения системы обыкновенных дифференциальных уравнений:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0$$

применяется следующий алгоритм на каждом шаге интегрирования от t_n к $t_{n+1} = t_n + h$:

$$\begin{aligned} k_1 &= f(t_n, y_n) \\ k_2 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \\ k_3 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \\ k_4 &= f\left(t_n + h, y_n + hk_3\right) \\ y_{n+1} &= y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{aligned}$$

Для системы уравнений в задаче оптимального управления:

$$\mathbf{y} = \begin{pmatrix} x_1 \\ x_2 \\ p_1 \\ p_2 \end{pmatrix}, \quad \mathbf{f}(t, \mathbf{y}) = \begin{pmatrix} x_2 \\ p_2 \\ -\frac{24}{1+e^{-\alpha t}} \\ -p_1 \end{pmatrix}$$

метод применяется к каждой компоненте вектора $\mathbf{y}$ одновременно. Этот процесс проделывается при некоторых значениях β_1, β_2. Затем мы меняем параметры β_1, β_2 так, чтобы минимизировать ошибку $x_1(1)^2 + p_2(1)^2$-$>min$

7 Метод Ньютона для решения нелинейной системы

Формулировка нелинейной системы

Краевая задача (10) преобразуется в задачу Коши путем выбора двух недостающих начальных условий, которые являются параметрами пристрелки:

$$\boldsymbol{\beta} = \begin{pmatrix} \beta_1 \\ \beta_2 \end{pmatrix}, \quad \text{где } \beta_1 = x_1(0), \quad \beta_2 = p_2(0).$$

После численного интегрирования задачи Коши (например, методом Рунге-Кутты 4-го порядка), мы проверяем выполнение граничных условий на правом конце $t = 1$. Это приводит к нелинейной системе двух уравнений относительно $\boldsymbol{\beta}$:

$$\mathbf{F}(\boldsymbol{\beta}) = \begin{pmatrix} F_1(\beta_1, \beta_2) \\ F_2(\beta_1, \beta_2) \end{pmatrix} = \begin{pmatrix} x_1(1; \beta_1, \beta_2) \\ p_2(1; \beta_1, \beta_2) \end{pmatrix} = \mathbf{0}, \tag{12}$$

где $x_1(1; \beta_1, \beta_2)$ и $p_2(1; \beta_1, \beta_2)$ — значения траекторий в $t = 1$, зависящие от выбора $\boldsymbol{\beta}$.

Итерационный процесс

Искомые параметры $\boldsymbol{\beta}$ находятся итерационным процессом Метода Ньютона. На каждой итерации k параметры обновляются по формуле:

$$\boldsymbol{\beta}^{(k+1)} = \boldsymbol{\beta}^{(k)} + \Delta\boldsymbol{\beta}^{(k)}. \tag{13}$$

Поправка $\Delta\boldsymbol{\beta}^{(k)}$ находится путем решения линейной системы:

$$\mathbf{J}(\boldsymbol{\beta}^{(k)})\Delta\boldsymbol{\beta}^{(k)} = -\mathbf{F}(\boldsymbol{\beta}^{(k)}). \tag{14}$$

Матрица Якоби

Матрица Якоби $\mathbf{J}$ представляет собой матрицу частных производных невязок $\mathbf{F}$ по параметрам $\boldsymbol{\beta}$:

$$\mathbf{J}(\boldsymbol{\beta}) = \begin{pmatrix} \frac{\partial F_1}{\partial \beta_1} & \frac{\partial F_1}{\partial \beta_2} \\ \frac{\partial F_2}{\partial \beta_1} & \frac{\partial F_2}{\partial \beta_2} \end{pmatrix} = \begin{pmatrix} \frac{\partial x_1(1)}{\partial \beta_1} & \frac{\partial x_1(1)}{\partial \beta_2} \\ \frac{\partial p_2(1)}{\partial \beta_1} & \frac{\partial p_2(1)}{\partial \beta_2} \end{pmatrix}. \tag{15}$$

На практике элементы этой матрицы вычисляются численно методом конечных разностей (конечных приращений). Например, элемент $\frac{\partial F_1}{\partial \beta_1}$ приближается как:

$$\frac{\partial F_1}{\partial \beta_1} \approx \frac{F_1(\beta_1 + \delta, \beta_2) - F_1(\beta_1, \beta_2)}{\delta},$$

где δ — малое приращение.

Итерации продолжаются до тех пор, пока норма невязки $||\mathbf{F}(\boldsymbol{\beta}^{(k)})||$ не станет меньше заданной погрешности ε.

Метод продолжения по параметру α

Для исследования задачи при разных $\alpha \in \{0.0, 0.1, 1.0, 5.1\}$ применяется метод продолжения. Найденное оптимальное решение $\boldsymbol{\beta}_{\text{опт}}$ для предыдущего значения α используется в качестве **начального приближения** $\boldsymbol{\beta}^{(0)}$ для следующего значения α. Это обеспечивает быструю сходимость метода Ньютона.

8 Аналитическое решение для $\alpha = 0$

При $\alpha = 0$ система упрощается:

$$\begin{cases} \dot{x}_1 = x_2, \\ \dot{x}_2 = p_2, \\ \dot{p}_1 = -12, \\ \dot{p}_2 = -p_1. \end{cases}$$

Из третьего уравнения: $p_1(t) = -12t + C_1$

Из четвертого: $p_2(t) = 6t^2 - C_1 t + C_2$

Из второго: $x_2(t) = 2t^3 - \frac{C_1}{2}t^2 + C_2 t + C_3$

Из первого: $x_1(t) = \frac{1}{2}t^4 - \frac{C_1}{6}t^3 + \frac{C_2}{2}t^2 + C_3 t + C_4$

Используем граничные условия:

$$
\begin{aligned}
x_2(0) &= C_3 = 0, \\
p_1(0) &= C_1 = 0, \\
p_2(1) &= 6 + C_2 = 0, \\
x_1(1) &= \frac{1}{2} - 3 + C_4 = 0
\end{aligned}
$$

Отсюда: $C_2 = -6$, $C_4 = \frac{5}{2}$
Решение:

$$x(t) = \frac{1}{2}t^4 - 3t^2 + \frac{5}{2}$$

9 Результаты программы

Таблица 1: Результаты численного эксперимента при $\alpha = 0.0$, $N = 1000$

t	$x_1(t)$	$x_2(t)$	$p_1(t)$	$p_2(t)$
0.000000	2.50000000	0.00000000	0.00000000	-6.00000000
0.200000	2.38080000	-1.18400000	-2.40000000	-5.76000000
0.400000	2.03280000	-2.27200000	-4.80000000	-5.04000000
0.600000	1.48480000	-3.16800000	-7.20000000	-3.84000000
0.800000	0.78480000	-3.77600000	-9.60000000	-2.16000000
1.000000	0.00000000	-4.00000000	-12.00000000	-0.00000000

Таблица 2: Результаты численного эксперимента при $\alpha = 0.1$, $N = 1000$

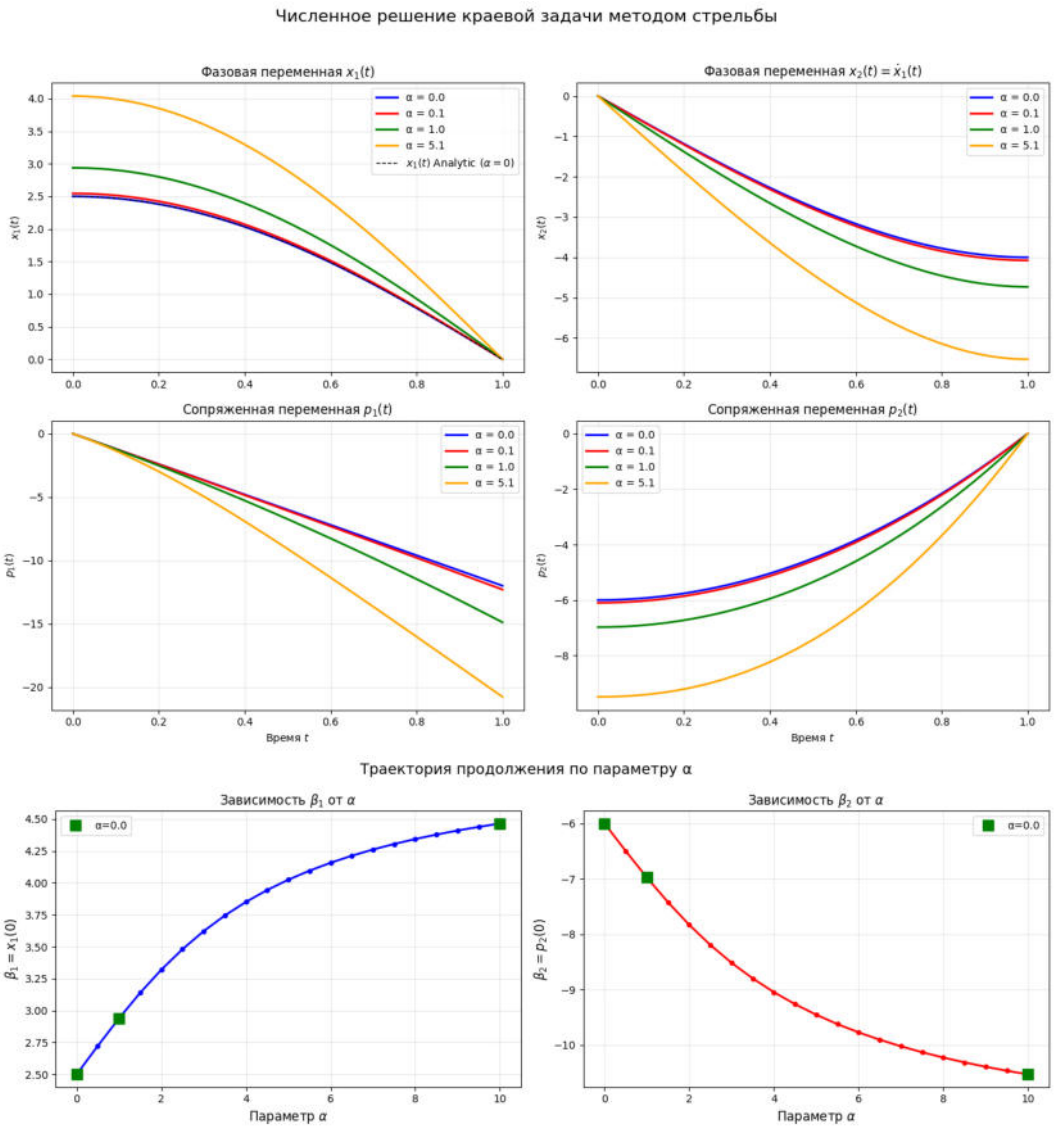
t	$x_1(t)$	$x_2(t)$	$p_1(t)$	$p_2(t)$
0.000000	2.54498810	0.00000000	0.00000000	-6.09997501
0.200000	2.42379020	-1.20395500	-2.41199980	-5.85917502
0.400000	2.06984130	-2.31135002	-4.84799680	-5.13357527
0.600000	1.51218138	-3.22474520	-7.30798380	-3.91837696
0.800000	0.79943437	-3.84574110	-9.79194882	-2.20878320
1.000000	0.00000000	-4.07497918	-12.29987508	-0.00000000

Таблица 3: Результаты численного эксперимента при $\alpha = 1.0$, $N = 1000$

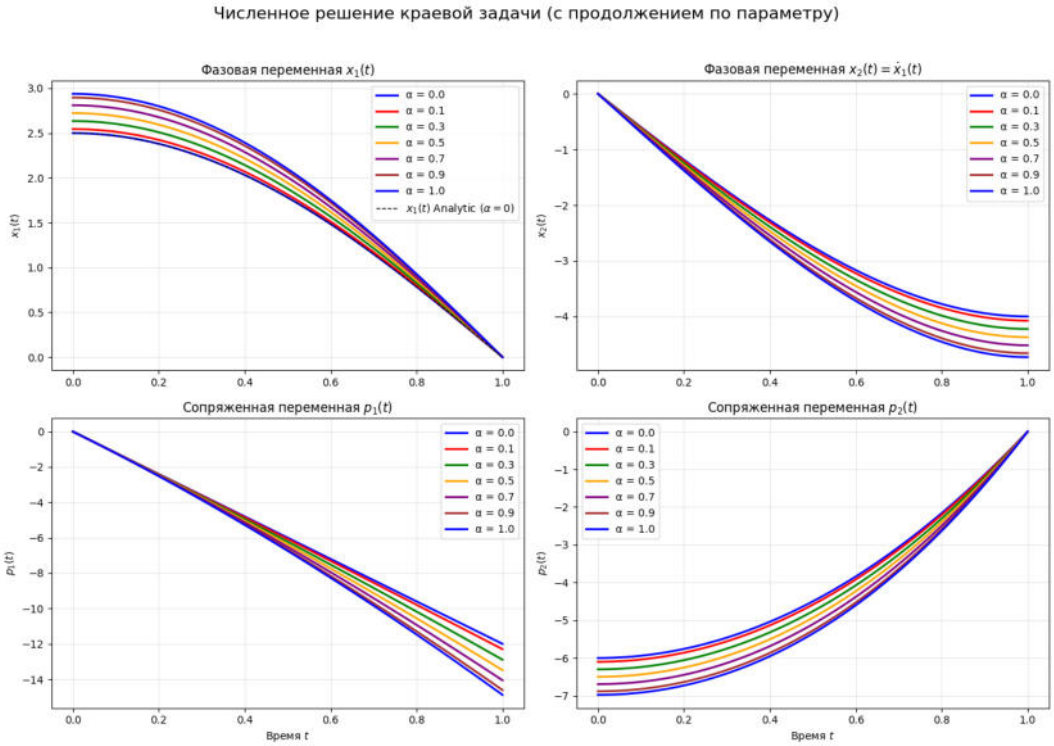
t	$x_1(t)$	$x_2(t)$	$p_1(t)$	$p_2(t)$
0.000000	2.93864159	0.00000000	0.00000000	-6.97612456
0.200000	2.79993509	-1.37882518	-2.51980053	-6.72813254
0.400000	2.39386266	-2.65606679	-5.27683372	-5.95237862
0.600000	1.75161067	-3.72146668	-8.26417848	-4.60203592
0.800000	0.92734307	-4.45556767	-11.47088365	-2.63207594
1.000000	0.00000000	-4.73014907	-14.88274817	-0.00000000

Таблица 4: Результаты численного эксперимента при $\alpha = 5.1$, $N = 1000$

t	$x_1(t)$	$x_2(t)$	$p_1(t)$	$p_2(t)$
0.000000	4.03963377	0.00000000	0.00000000	-9.49142231
0.200000	3.85068595	-1.88027858	-2.98717590	-9.21163397
0.400000	3.29561476	-3.63791032	-6.91339228	-8.23355180
0.600000	2.41402462	-5.11732301	-11.35376227	-6.41240278
0.800000	1.27899069	-6.14186904	-16.01703033	-3.67754457
1.000000	0.00000000	-6.52545860	-20.76673438	-0.00000000



10 Продолжение по параметру



11 Вывод

Зависимость параметров β от α.

Параметры $\beta_1 = x_1(0)$ и $\beta_2 = p_2(0)$, выбранные в качестве параметров пристрелки, демонстрируют монотонную зависимость от параметра α:

- $\beta_1 = x_1(0)$**:** С увеличением параметра α от 0.0 до 5.1, начальное значение фазовой переменной $x_1(0)$ монотонно увеличивается. Например, при $\alpha = 0.0$, $\beta_1 \approx 2.500$, а при $\alpha = 5.1$, $\beta_1 \approx 4.040$.

- $\beta_2 = p_2(0)$**:** С увеличением параметра α от 0.0 до 5.1, начальное значение сопряженной переменной $p_2(0)$ монотонно уменьшается (становится более отрицательным).

Увеличение параметра α приводит к увеличению начального состояния $x_1(0)$ и увеличению абсолютной величины скорости $\dot{x}_1(t) = x_2(t)$ для достижения требуемого конечного состояния $x_1(1) = 0$.

12 Литература

1. **Алексеев В. М., Галеев Э. М., Тихомирова В. М.** Сборник задач по оптимизации. Теория. Примеры. Задачи. Учебное пособие. — М.: Наука. Главная редакция физико-математической литературы, 1984. — 288 с.

2. **Бахвалов Н. С., Жидков Н. П., Кобельков Г. М.** Численные методы [Электронный ресурс] / Н. С. Бахвалов, Н. П. Жидков, Г. М. Кобельков. — 8-е изд. (эл.). — М. : БИНОМ. Лаборатория знаний, 2015. — (Классический университетский учебник).

3. **Григорьев И. С.** Методическое пособие по численным методам решения краевых задач принципа максимума в задачах оптимального управления. — М.: Издательство Центра прикладных исследований при механико-математическом факультете МГУ, 2005. — 160 с.

4. **Ортега Дж., Пул У.** Введение в численные методы решения дифференциальных уравнений / Пер. с англ.; Под ред. А. А. Абрамова. — М. : Наука. Гл. ред. физ.-мат. лит., 1986 — 288 с.

13 Листинг программы

```
import numpy as np
import matplotlib.pyplot as plt

class VariationalSolver:
    def __init__(self, alpha_values, lambda0=0.5, step_size=0.1,
   use_continuation=True):
        self.alpha_values = sorted(alpha_values)
        self.lambda0 = lambda0
        self.step_size = step_size
        self.use_continuation = use_continuation
        self.results = {}
        self.continuation_results = {}
    def ode_system(self, t, y, alpha):
        x1, x2, p1, p2 = y

        f1 = x2
        f2 = p2

        f3 = -24.0 / (1.0 + np.exp(-alpha * t))
        f4 = -p1
        return np.array([f1, f2, f3, f4])

    def integrate_system(self, beta1, beta2, alpha, n_points=1000):
        t_start, t_end = 0.0, 1.0
        h = (t_end - t_start) / n_points
        t_values = np.linspace(t_start, t_end, n_points + 1)

        y_current = np.array([beta1, 0.0, 0.0, beta2])
        y_values = np.zeros((4, n_points + 1))
        y_values[:, 0] = y_current

        for i in range(n_points):
            t = t_values[i]

            k1 = h * self.ode_system(t, y_current, alpha)
            k2 = h * self.ode_system(t + h/2.0, y_current + k1/2.0, alpha)
            k3 = h * self.ode_system(t + h/2.0, y_current + k2/2.0, alpha)
            k4 = h * self.ode_system(t + h, y_current + k3, alpha)

            y_current += (k1 + 2.0*k2 + 2.0*k3 + k4) / 6.0
            y_values[:, i+1] = y_current

        return t_values, y_values

    def boundary_residuals(self, beta, alpha):
        beta1, beta2 = beta
        t_vals, y_vals = self.integrate_system(beta1, beta2, alpha)

        residual1 = y_vals[0, -1] # x1(1)
        residual2 = y_vals[3, -1] # p2(1)
        return np.array([residual1, residual2])

    def compute_jacobian(self, beta, alpha, delta=1e-8):
        n = len(beta)
```

```python
        J = np.zeros((n, n))
        F0 = self.boundary_residuals(beta, alpha)

        for j in range(n):
            beta_perturbed = beta.copy()
            beta_perturbed[j] += delta
            F_perturbed = self.boundary_residuals(beta_perturbed, alpha)
            J[:, j] = (F_perturbed - F0) / delta
        return J

    def solve_parameters(self, alpha, initial_guess=None, max_iter=20, tol=1e-8):
        if initial_guess is None:
            initial_guess = np.array([5.0, -6.0])

        beta = initial_guess.copy()

        for iteration in range(max_iter):
            F = self.boundary_residuals(beta, alpha)
            residual_norm = np.linalg.norm(F)

            if residual_norm < tol:
                return beta

            J = self.compute_jacobian(beta, alpha)

            try:
                delta_beta = np.linalg.solve(J, -F)
            except np.linalg.LinAlgError:
                delta_beta = -np.linalg.pinv(J) @ F

            beta += delta_beta

        return beta

    def solve_with_continuation(self, alpha_start=0.0, alpha_end=10.0):

        print(f"\n=== ===")
        print(f"              : {self.step_size}")
        print(f"                :      {alpha_start}      {alpha_end}")

        num_steps = int((alpha_end - alpha_start) / self.step_size) + 1
        alphas_continuation = np.linspace(alpha_start, alpha_end, num_steps)

        current_beta = np.array([5.0, -6.0])  #          =0: x1(0)=5, p2(0)=-6

        for i, alpha in enumerate(alphas_continuation):
            print(f"\ n       {i+1}/{num_steps}:    = {alpha:.3f}")

            if i == 0:
                beta_opt = self.solve_parameters(alpha, initial_guess=current_beta)
            else:
                beta_opt = self.solve_parameters(alpha, initial_guess=current_beta)

            self.continuation_results[alpha] = beta_opt

            current_beta = beta_opt.copy()
```

```
            print(f"                    :  1  = {beta_opt[0]:.6f},  2  = {beta_opt[1]:.6f}")

        return self.continuation_results

    def solve_for_target_alphas(self):
        print(f"\n===                                                    ===")

        if self.use_continuation:
            alpha_min = min(self.alpha_values)
            alpha_max = max(self.alpha_values)

            self.solve_with_continuation(alpha_start=alpha_min, alpha_end=alpha_max)

        for alpha in self.alpha_values:
            print(f"\n---                          = {alpha} ---")

            if self.use_continuation and alpha in self.continuation_results:
                initial_guess = self.continuation_results[alpha]
                print(f"                                          ")
            else:
                initial_guess = self._find_closest_initial_guess(alpha)

            beta_opt = self.solve_parameters(alpha, initial_guess=initial_guess)
            beta1_opt, beta2_opt = beta_opt

            t_vals, y_vals = self.integrate_system(beta1_opt, beta2_opt, alpha)

            self.results[alpha] = {
                'beta1': beta1_opt,
                'beta2': beta2_opt,
                'time': t_vals,
                'x1': y_vals[0],
                'x2': y_vals[1],
                'p1': y_vals[2],
                'p2': y_vals[3],
                'final_x1': y_vals[0, -1],
                'final_p2': y_vals[3, -1],
                'initial_guess_used': initial_guess
            }

            print(f"   1  = {beta1_opt:.8f},  2  = {beta2_opt:.8f}")

        return self.results

    def _find_closest_initial_guess(self, alpha):
        if not self.continuation_results:
            return np.array([5.0, -6.0])

        available_alphas = sorted(self.continuation_results.keys())

        closest_alpha = None
        for a in reversed(available_alphas):
            if a <= alpha:
                closest_alpha = a
```

```
                break

        if closest_alpha is not None:
            print(f"                                                    ={closest_alpha}                                                    ")
            return self.continuation_results[closest_alpha].copy()
        else:
            print(f"                                                    ={available_alphas[0]}                                                    ")
            return self.continuation_results[available_alphas[0]].copy()

    def display_results(self, n_points_table=6):
        print("\n" + "="*85)
        print("                                        ")

        for alpha in self.alpha_values:
            res = self.results[alpha]

            print("\n" + "="*85)
            print(f"                                = {alpha}")
            print(f" 1  = x1(0)     {res['beta1']:.8f},  2  = p2(0)     {res['beta2']:.8f}")
            print("="*85)

            indices = np.linspace(0, len(res['time']) - 1, n_points_table, dtype=int)

            print(f"{'t':<10} {'x1(t)':<15} {'x2(t)':<15} {'p1(t)':<15} {'p2(t)':<15}")
            print("-" * 75)

            for i in indices:
                t = res['time'][i]
                x1 = res['x1'][i]
                x2 = res['x2'][i]
                p1 = res['p1'][i]
                p2 = res['p2'][i]
                print(f"{t:.6f} {x1:.8f} {x2:.8f} {p1:.8f} {p2:.8f}")
            print("-" * 75)
            print("                                                    :")
            print(f"x1(1) = {res['final_x1']:.2e} (                    0)")
            print(f"p2(1) = {res['final_p2']:.2e} (                    0)")

    def plot_continuation_path(self):
        if not self.continuation_results:
            print("                    .")
            return

        alphas = sorted(self.continuation_results.keys())
        beta1_vals = [self.continuation_results[a][0] for a in alphas]
        beta2_vals = [self.continuation_results[a][1] for a in alphas]

        fig, axes = plt.subplots(1, 2, figsize=(14, 5))

        axes[0].plot(alphas, beta1_vals, 'b-o', linewidth=2, markersize=4)
        axes[0].set_xlabel(r'                $\alpha$', fontsize=12)
        axes[0].set_ylabel(r'$\beta_1 = x_1(0)$', fontsize=12)
        axes[0].set_title(r'                        $\beta_1$      $\alpha$')
        axes[0].grid(True, alpha=0.3)
```

```python

        axes[1].plot(alphas, beta2_vals, 'r-o', linewidth=2, markersize=4)
        axes[1].set_xlabel(r'                    $\alpha$', fontsize=12)
        axes[1].set_ylabel(r'$\beta_2 = p_2(0)$', fontsize=12)
        axes[1].set_title(r'                        $\beta_2$      $\alpha$')
        axes[1].grid(True, alpha=0.3)

        for alpha in self.alpha_values:
            if alpha in self.continuation_results:
                beta1 = self.continuation_results[alpha][0]
                beta2 = self.continuation_results[alpha][1]
                axes[0].plot(alpha, beta1, 'gs', markersize=10, label=f'  ={alpha}' if alpha == self.alpha_values[0] else "")
                axes[1].plot(alpha, beta2, 'gs', markersize=10, label=f'  ={alpha}' if alpha == self.alpha_values[0] else "")

        axes[0].legend()
        axes[1].legend()
        plt.suptitle('                        ', fontsize=14)
        plt.tight_layout()
        plt.show()

    def plot_solutions(self):
        fig, axes = plt.subplots(2, 2, figsize=(14, 10))
        colors = ['blue', 'red', 'green', 'orange', 'purple', 'brown']

        t_analytic = np.linspace(0, 1, 100)
        x1_analytic = 0.5*t_analytic**4 - 3*t_analytic**2 + 5/2

        for idx, alpha in enumerate(self.alpha_values):
            res = self.results[alpha]
            color = colors[idx % len(colors)]
            t = res['time']

            axes[0,0].plot(t, res['x1'], color=color, linewidth=2, label=f'   ={alpha}')

            axes[0,1].plot(t, res['x2'], color=color, linewidth=2, label=f'   ={alpha}')

            axes[1,0].plot(t, res['p1'], color=color, linewidth=2, label=f'   ={alpha}')

            axes[1,1].plot(t, res['p2'], color=color, linewidth=2, label=f'   ={alpha}')

        if 0.0 in self.alpha_values:
             axes[0,0].plot(t_analytic, x1_analytic, color='black', linestyle='--',
                            linewidth=1, label=r'$x_1(t)$ Analytic ($\alpha=0$)')

        axes[0,0].set_title(r'                                        $x_1(t)$')
        axes[0,0].set_ylabel(r'$x_1(t)$')
        axes[0,0].legend()

        axes[0,1].set_title(r'                                        $x_2(t) = \dot{x}_1(t)$')
        axes[0,1].set_ylabel(r'$x_2(t)$')
```

```
        axes[0,1].legend()

        axes[1,0].set_title(r'                                                    $p_1(t)$')
        axes[1,0].set_ylabel(r'$p_1(t)$')
        axes[1,0].set_xlabel(r'            $t$')
        axes[1,0].legend()

        axes[1,1].set_title(r'                                                    $p_2(t)$')
        axes[1,1].set_ylabel(r'$p_2(t)$')
        axes[1,1].set_xlabel(r'            $t$')
        axes[1,1].legend()

        for ax in axes.flat:
            ax.grid(True, alpha=0.3)

        method_name = "                                                    " if self.use_continuation else "                                    "
        plt.suptitle(f'                ({method_name})', fontsize=16)
        plt.tight_layout(rect=[0, 0, 1, 0.96])
        plt.show()

if __name__ == "__main__":
    print("="*70)
    print("                    ")
    print("                                                            ")
    print("="*70)

    print("\n===                    1:                          ===")
    alphas = [0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 9.5, 9.9, 9.95, 10.0, 10.1, 10.2]  #                    =10.0

    solver1 = VariationalSolver(alphas, step_size=1.0, use_continuation=True)
    solver1.solve_for_target_alphas()
    solver1.display_results()
    solver1.plot_continuation_path()
    solver1.plot_solutions()

    print("\n\n===                    2:                    (                          ) ===")
    solver2 = VariationalSolver(alphas, use_continuation=False)
    solver2.solve_for_target_alphas()
    solver2.display_results()
    solver2.plot_solutions()

    print("\n\n===                    3:                                                =10                                    =9 ===")
    solver3 = VariationalSolver([10.0], step_size=1.0, use_continuation=True)

    print("                                                            =9...")
    beta_for_9 = solver3.solve_parameters(9.0, initial_guess=np.array([5.0, -6.0]))
    print(f"                        =9:  1  = {beta_for_9[0]:.6f},  2  = {beta_for_9[1]:.6f}")

    print("\ n
```

```
                                        =10...")
    beta_for_10 = solver3.solve_parameters(10.0, initial_guess=beta_for_9)
    print(f"                            =10:  1  = {beta_for_10[0]:.6f},  2  = {beta_for_10[1]:.6f}")

    F = solver3.boundary_residuals(beta_for_10, 10.0)
    print(f"                                                                =10: x1(1) = {F[0]:.2e}, p2(1) = {F[1]:.2e}")
```